

Product Requirements Document

Project: FormatShift Web Application

1. Executive Summary

FormatShift is a web-based utility application designed to help developers, students, and professionals instantly transform text and data formats. The tool will provide a frictionless, one-click interface to sanitize, convert, and format strings and structured data without requiring the user to write custom scripts.

2. Problem Statement

Users frequently encounter formatting mismatches when moving data between different systems or documents. Common tasks like converting Markdown to HTML, prettifying unreadable JSON, or extracting URLs from a block of text require either writing throwaway scripts or using multiple, ad-laden web utilities. There is a need for a clean, unified, and fast tool that handles these tasks seamlessly in the browser.

3. Product Scope & Phasing

The product development lifecycle is divided into two phases to ensure a rapid launch and iterative improvement.

Phase 1: Minimal Viable Product (MVP) - Client-Side Only

- Core text manipulation (Case conversion, text cleaning).
- Developer utilities (JSON formatting, Markdown rendering).
- Browser-based processing ensuring 100% data privacy.
- Clipboard integration (Copy/Paste actions).

Phase 2: Advanced Capabilities - Server-Side Processing

- File upload/download functionality (CSV to Excel, Word DOCX cleanup).

- Batch processing of large datasets.
- User accounts for saving custom regex rules or formatting macros.

4. Functional Requirements (Phase 1)

Feature ID	Feature Name	Description	Priority
FR-01	Text Input Area	A large textarea where users can paste raw text or data. Must support line numbers for code formats.	High
FR-02	String Manipulation	Buttons to convert text to UPPERCASE, lowercase, Title Case, and camelCase.	High
FR-03	JSON Formatter	Parse raw JSON strings and output pretty-printed, indented JSON. Must handle syntax errors gracefully.	High
FR-04	Data Extraction	Regex-powered tools to extract all Email addresses, URLs, or IP addresses from the input text into a list.	Medium
FR-05	Markdown to HTML	Convert raw markdown text into properly formatted HTML code blocks.	Medium
FR-06	One-Click Copy	A dedicated UI button to instantly copy the transformed output to the system clipboard.	High

5. Technical Specifications

Architecture Approach: The application will utilize a decoupled architecture. The MVP will rely entirely on the frontend, with a scalable backend introduced in Phase 2 for heavy processing.

- **Frontend ([React](#)):** React is chosen for its component-based architecture, making it easy to create modular formatting tools. State management will handle the dynamic updating of the output area as inputs change.
- **Backend ([Python](#) / [FastAPI](#)):** Reserved for Phase 2. Python provides robust libraries (e.g., Pandas for CSVs, BeautifulSoup for HTML parsing) that are ideal for complex data transformations that cannot run efficiently in the browser.

- **Deployment:** Vercel or Netlify for seamless CI/CD of the frontend application.

6. UI/UX Guidelines

The interface must prioritize usability and speed. Key design principles include:

- **Split-Pane Layout:** A side-by-side layout (or top/bottom on mobile) featuring the Input on the left and Output on the right.
- **Immediate Feedback:** Formats should update automatically in real-time as the user types or clicks a formatting button (avoiding unnecessary "Submit" clicks).
- **Minimalist Aesthetics:** Use plenty of whitespace, clear typography, and subtle visual cues (like a green checkmark animation when "Copied to Clipboard" is triggered).
- **Accessibility:** Ensure high contrast for text areas and full keyboard navigability for power users.

7. Success Metrics

- **Performance:** Time to interactive (TTI) under 1.5 seconds.
- **Reliability:** Zero crashes when handling large strings (up to 1MB of text).
- **User Engagement:** Average session duration indicating successful task completion (e.g., >30 seconds).